

Universidade Federal de Minas Gerais

Departamento de Engenharia Eletrônica – DELT

Engenharia de Controle e Automação

Trabalho Prático 2

Filtro de Bayes e Estimação Recursiva

Pedro Antonio de Souza Campos Rodrigues

EEE935 – Robótica Probabilística

Belo Horizonte – MG

Agosto de 2026

Sumário

1	Estado e hipótese de Markov	2
2	Independência condicional	2
3	Estrutura do filtro de Bayes	3
4	Interpretação	3
5	Atualização Bayesiana	3
6	Predição discreta	4
7	Update discreto	5
8	Predição como convolução	5
8.1	Código da questão 8	6
9	Update: likelihood estreita e larga	8
9.1	Código da questão 9	9
10	Simulação completa de um filtro de Bayes 1D	10
10.1	Código da questão 10	11

1. Estado e hipótese de Markov

Enunciado. Considere os modelos

$$\mathbf{x}_t = (x, y, \theta) \quad \text{e} \quad \mathbf{x}_t = (x, y, \theta, v, \omega, b_g).$$

Qual deles satisfaz melhor a hipótese de Markov? O que ocorre se o bias b_g não for modelado?

Solução. O segundo modelo satisfaz melhor a hipótese de Markov, pois guarda mais informações sobre o estado atual do robô. Além da posição e orientação, ele inclui as velocidades e o bias do giroscópio.

A ideia da hipótese de Markov é simples: conhecendo o estado atual, não deveria ser necessário conhecer todo o passado para prever o próximo estado.

Se b_g não for modelado, o erro do giroscópio pode ser interpretado como uma mudança real de orientação. Com o tempo, esse erro se acumula e a estimativa pode apresentar **deriva**.

2. Independência condicional

Enunciado. Explique o significado de

$$p(\mathbf{x}_t \mid \mathbf{x}_{0:t-1}, \mathbf{u}_{1:t}, \mathbf{z}_{1:t-1}) = p(\mathbf{x}_t \mid \mathbf{x}_{t-1}, \mathbf{u}_t)$$

e

$$p(\mathbf{z}_t \mid \mathbf{x}_{0:t}, \mathbf{z}_{1:t-1}, \mathbf{u}_{1:t}) = p(\mathbf{z}_t \mid \mathbf{x}_t).$$

Solução. A primeira expressão diz que, para prever o novo estado, basta conhecer o estado anterior e o comando aplicado. O restante do histórico não é necessário.

Isso pode deixar de ser verdade se o estado não guardar alguma informação importante, como velocidade, aceleração, bias de sensor ou derrapagem.

A segunda expressão diz que a medição atual depende apenas do estado atual. Por exemplo, sabendo onde o robô está, o modelo do sensor calcula a probabilidade de observar aquela medição.

Essa hipótese pode ser violada quando o sensor possui memória, atraso, bias não modelado ou ruído correlacionado no tempo.

3. Estrutura do filtro de Bayes

Enunciado. Explique o papel dos termos em

$$\text{bel}(\mathbf{x}_t) = \eta p(\mathbf{z}_t | \mathbf{x}_t) \int p(\mathbf{x}_t | \mathbf{u}_t, \mathbf{x}_{t-1}) \text{bel}(\mathbf{x}_{t-1}) d\mathbf{x}_{t-1}.$$

Solução. O filtro de Bayes possui duas etapas: **predição** e **correção**.

- $\text{bel}(\mathbf{x}_{t-1})$: o que acreditávamos sobre o estado no instante anterior.
- $p(\mathbf{x}_t | \mathbf{u}_t, \mathbf{x}_{t-1})$: modelo de movimento. Diz como o robô pode mudar de estado ao aplicar o controle $\mathbf{u}_t$.
- A integral produz o **belief predito**, antes de usar a nova medição.
- $p(\mathbf{z}_t | \mathbf{x}_t)$: likelihood. Mede o quanto cada estado combina com a medição recebida.
- η : constante usada para normalizar o resultado, fazendo a probabilidade total ser igual a 1.

Assim, primeiro o filtro prevê onde o robô pode estar e depois usa a medição para corrigir essa previsão.

4. Interpretação

Enunciado. Explique por que a predição aumenta a entropia e a correção reduz a entropia.

Solução. Na **predição**, existe incerteza no movimento. Por isso, depois que o robô se move, existem mais posições possíveis para ele. O belief tende a ficar mais espalhado e a entropia aumenta.

Na **correção**, uma nova medição traz informação. Os estados compatíveis com a medição ganham mais probabilidade e os demais perdem. Assim, o belief tende a ficar mais concentrado e a entropia diminui.

Em resumo: **o movimento adiciona incerteza e a medição ajuda a reduzi-la.**

5. Atualização Bayesiana

Enunciado. Considere

$$P(\text{open}) = 0.5, \quad P(z | \text{open}) = 0.7, \quad P(z | \text{closed}) = 0.2.$$

Calcule $P(\text{open} \mid z)$.

Solução. Como só existem os estados *open* e *closed*,

$$P(\text{closed}) = 0.5.$$

Aplicando Bayes,

$$P(\text{open} \mid z) = \frac{0.7 \cdot 0.5}{0.7 \cdot 0.5 + 0.2 \cdot 0.5} = \frac{0.35}{0.45} \approx 0.778.$$

Portanto,

$$P(\text{open} \mid z) \approx 0.778.$$

Após observar z , a probabilidade de o estado ser *open* sobe de 50% para aproximadamente 77.8%.

6. Predição discreta

Enunciado. Considere

$$x \in \{1, 2, 3\}, \quad \mathbf{b}_{t-1} = \begin{bmatrix} 0.2 & 0.6 & 0.2 \end{bmatrix}^T,$$

com probabilidade 0.2 de permanecer no estado atual e 0.8 de mover +1.

Solução. O enunciado não informa o que acontece ao tentar avançar a partir do estado 3. Aqui será considerada **saturação**: se o sistema já está em 3, ele permanece em 3.

A matriz de transição fica

$$T = \begin{bmatrix} 0.2 & 0 & 0 \\ 0.8 & 0.2 & 0 \\ 0 & 0.8 & 1 \end{bmatrix}.$$

Então,

$$\bar{\mathbf{b}}_t = T\mathbf{b}_{t-1} = \begin{bmatrix} 0.04 \\ 0.28 \\ 0.68 \end{bmatrix}.$$

Logo,

$$\bar{\mathbf{b}}_t = \begin{bmatrix} 0.04 & 0.28 & 0.68 \end{bmatrix}^T.$$

A maior probabilidade fica no estado 3, pois grande parte da massa do belief se desloca para a direita.

7. Update discreto

Enunciado. Para o belief predito da questão anterior, considere

$$\mathbf{l} = \begin{bmatrix} 0.3 & 0.9 & 0.4 \end{bmatrix}^T.$$

Calcule o posterior.

Solução. Primeiro multiplicamos cada valor do belief pela likelihood correspondente:

$$\tilde{\mathbf{b}}_t = \begin{bmatrix} 0.3 \cdot 0.04 \\ 0.9 \cdot 0.28 \\ 0.4 \cdot 0.68 \end{bmatrix} = \begin{bmatrix} 0.012 \\ 0.252 \\ 0.272 \end{bmatrix}.$$

A soma é 0.536. Dividindo todos os valores por essa soma,

$$\mathbf{b}_t \approx \begin{bmatrix} 0.0224 \\ 0.4701 \\ 0.5075 \end{bmatrix}.$$

Portanto,

$$\mathbf{b}_t \approx \begin{bmatrix} 0.0224 & 0.4701 & 0.5075 \end{bmatrix}^T.$$

Depois da medição, os estados 2 e 3 ficam com probabilidades próximas, enquanto o estado 1 se torna muito pouco provável.

8. Predição como convolução

Enunciado. Implemente

$$x_t = x_{t-1} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2),$$

e plote o belief anterior, o kernel e o belief predito.

Solução. Foi usado um belief inicial com dois picos e um ruído gaussiano com $\sigma = 0.70$. A predição é feita pela convolução entre o belief anterior e o ruído de movimento.

Na prática, essa convolução **espalha** a distribuição. Isso representa o fato de que, após um movimento com ruído, ficamos menos certos sobre a posição exata do robô.

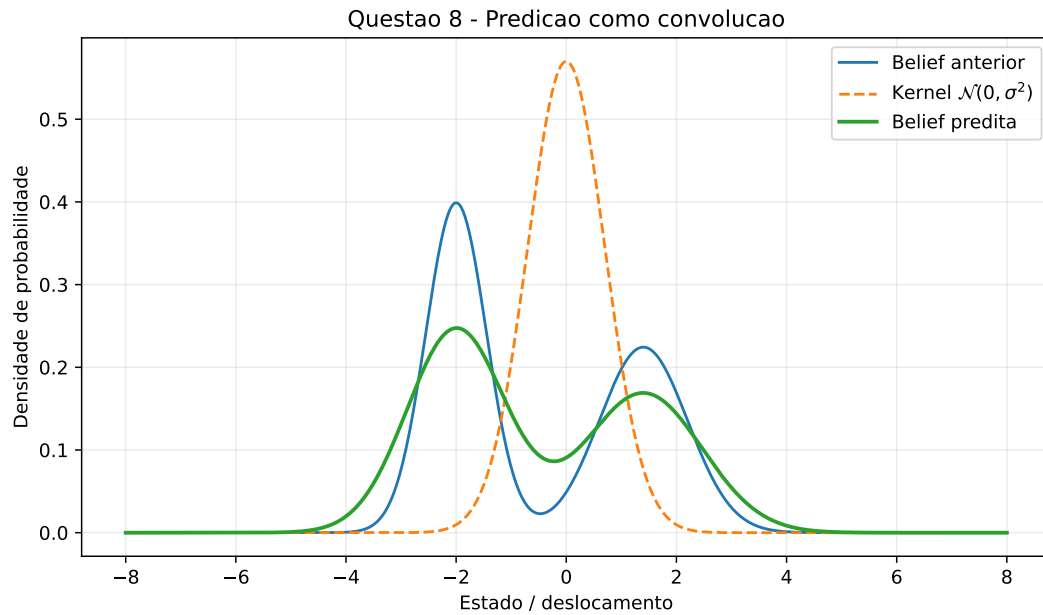


Figura 1: Belief anterior, kernel gaussiano e belief predito.

As distribuições foram normalizadas numericamente, com área aproximadamente igual a 1.

8.1. Código da questão 8

```

1  """Questao 8 - Predicao como convolucao.
2
3  Modelo:  $x_t = x_{t-1} + \epsilon$ ,  $\epsilon \sim N(0, \sigma^2)$ .
4  O script plota o belief anterior, o kernel gaussiano e o belief predito.
5  """
6
7  from pathlib import Path
8
9  import matplotlib.pyplot as plt
10 import numpy as np
11
12
13 OUT_DIR = Path(__file__).resolve().parents[1] / "imagens"
14 OUT_DIR.mkdir(parents=True, exist_ok=True)
15
16
17 def gaussian(x: np.ndarray, mean: float, std: float) -> np.ndarray:
18     """Gaussian probability density function."""
19     return np.exp(-0.5 * ((x - mean) / std) ** 2) / (np.sqrt(2.0 * np.pi) * std)
20
21
22 def normalize_density(p: np.ndarray, dx: float) -> np.ndarray:
23     """Normalize a sampled density so its numerical integral is one."""

```

```
24     return p / (np.sum(p) * dx)
25
26
27 # Spatial grid.
28 x = np.linspace(-8.0, 8.0, 1601)
29 dx = x[1] - x[0]
30
31 # A bimodal prior is used only to make the smoothing effect easy to see.
32 bel_prior = 0.55 * gaussian(x, -2.0, 0.55) + 0.45 * gaussian(x, 1.4, 0.80)
33 bel_prior = normalize_density(bel_prior, dx)
34
35 # Motion-noise kernel: epsilon ~ N(0, sigma^2).
36 sigma = 0.70
37 offsets = (np.arange(x.size) - x.size // 2) * dx
38 kernel = gaussian(offsets, 0.0, sigma)
39 kernel = normalize_density(kernel, dx)
40
41 # Prediction: bel_bar(x_t) = integral p(x_t/x_{t-1}) bel(x_{t-1}) dx_{t-1}.
42 bel_pred = np.convolve(bel_prior, kernel, mode="same") * dx
43 bel_pred = normalize_density(bel_pred, dx)
44
45 # Plot. The kernel is drawn against displacement, but shares the same horizontal scale.
46
47 fig, ax = plt.subplots(figsize=(8.0, 4.8))
48 ax.plot(x, bel_prior, label="Belief anterior")
49 ax.plot(offsets, kernel, linestyle="--", label=r"Kernel $\mathcal{N}(0, \sigma^2)$")
50 ax.plot(x, bel_pred, linewidth=2.0, label="Belief predita")
51 ax.set_xlabel("Estado / deslocamento")
52 ax.set_ylabel("Densidade de probabilidade")
53 ax.set_title("Questao 8 - Predicao como convolucao")
54 ax.grid(True, alpha=0.25)
55 ax.legend()
56 fig.tight_layout()
57 fig.savefig(OUT_DIR / "questao8_convolucao.pdf", bbox_inches="tight")
58 fig.savefig(OUT_DIR / "questao8_convolucao.png", dpi=220, bbox_inches="tight")
59 plt.close(fig)
60
61 print(f"Integral prior:      {np.sum(bel_prior) * dx:.6f}")
62 print(f"Integral kernel:      {np.sum(kernel) * dx:.6f}")
63 print(f"Integral predicted: {np.sum(bel_pred) * dx:.6f}")
```

Listing 1: Predição como convolução.

9. Update: likelihood estreita e larga

Enunciado. Implemente

$$\text{bel}(x_t) = \eta p(z_t | x_t) \overline{\text{bel}}(x_t)$$

e compare uma likelihood estreita com uma larga.

Solução. Foi usado o mesmo belief predito e a mesma medição $z = 1.50$ nos dois casos. A única diferença foi a incerteza da medição:

$$\sigma_z = 0.45 \quad (\text{estreita}), \quad \sigma_z = 1.80 \quad (\text{larga}).$$

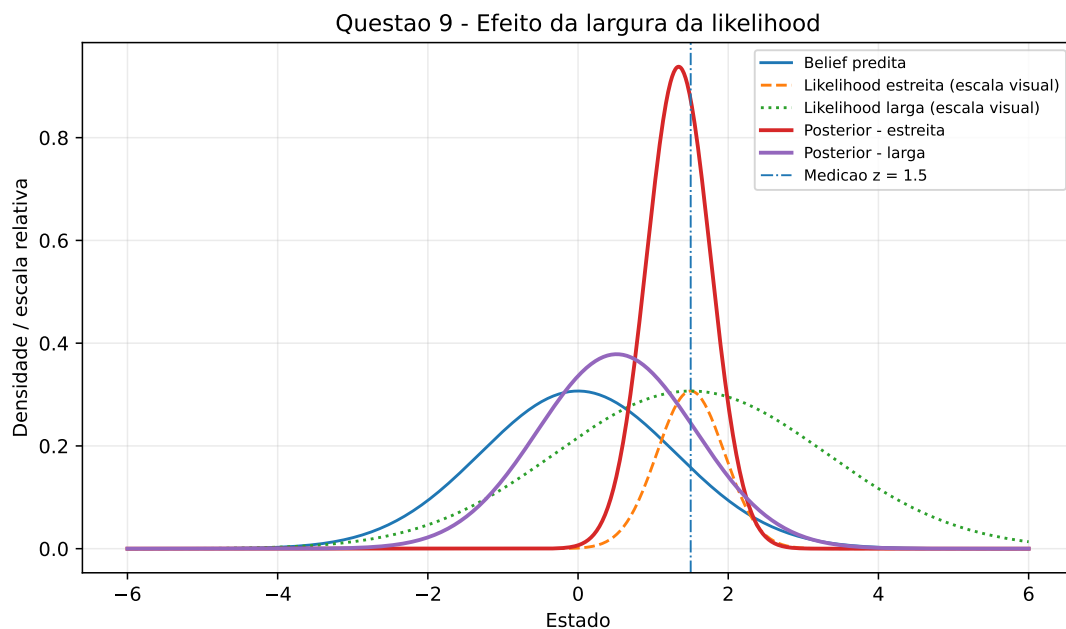


Figura 2: Update com uma likelihood estreita e uma likelihood larga.

Os resultados foram:

Distribuição	Média	Desvio-padrão
Belief predito	0.0000	1.2999
Posterior – likelihood estreita	1.3395	0.4252
Posterior – likelihood larga	0.5142	1.0539

A likelihood estreita representa um sensor mais preciso. Por isso, ela puxa mais o posterior para perto da medição e reduz bastante a incerteza.

A likelihood larga representa uma medição menos precisa. Nesse caso, o filtro confia menos nela e o posterior continua mais próximo do belief anterior.

9.1. Código da questão 9

```
1 """Questao 9 - Update Bayesiano com likelihood estreita e larga."""
2
3 from pathlib import Path
4
5 import matplotlib.pyplot as plt
6 import numpy as np
7
8
9 OUT_DIR = Path(__file__).resolve().parents[1] / "imagens"
10 OUT_DIR.mkdir(parents=True, exist_ok=True)
11
12
13 def gaussian(x: np.ndarray, mean: float, std: float) -> np.ndarray:
14     return np.exp(-0.5 * ((x - mean) / std) ** 2) / (np.sqrt(2.0 * np.pi) * std)
15
16
17 def normalize_density(p: np.ndarray, dx: float) -> np.ndarray:
18     return p / (np.sum(p) * dx)
19
20
21 def moments(x: np.ndarray, p: np.ndarray, dx: float) -> tuple[float, float]:
22     mean = np.sum(x * p) * dx
23     var = np.sum((x - mean) ** 2 * p) * dx
24     return float(mean), float(np.sqrt(var))
25
26
27 x = np.linspace(-6.0, 6.0, 1601)
28 dx = x[1] - x[0]
29
30 # Predicted belief used before the measurement update.
31 bel_pred = gaussian(x, mean=0.0, std=1.30)
32 bel_pred = normalize_density(bel_pred, dx)
33
34 # Same measurement, two different sensor-noise levels.
35 z = 1.50
36 lik_narrow = gaussian(x, mean=z, std=0.45)
37 lik_wide = gaussian(x, mean=z, std=1.80)
38
39 # Bayes update:  $bel(x_t) = \eta p(z_t/x_t) bel\_bar(x_t)$ .
40 post_narrow = normalize_density(lik_narrow * bel_pred, dx)
41 post_wide = normalize_density(lik_wide * bel_pred, dx)
42
43 mean_prior, std_prior = moments(x, bel_pred, dx)
44 mean_narrow, std_narrow = moments(x, post_narrow, dx)
45 mean_wide, std_wide = moments(x, post_wide, dx)
```

```

46
47 fig, ax = plt.subplots(figsize=(8.0, 4.8))
48 ax.plot(x, bel_pred, label="Belief predita")
49 ax.plot(x, lik_narrow / np.max(lik_narrow) * np.max(bel_pred), linestyle="--", label="
    Likelihood estreita (escala visual)")
50 ax.plot(x, lik_wide / np.max(lik_wide) * np.max(bel_pred), linestyle=":", label="
    Likelihood larga (escala visual)")
51 ax.plot(x, post_narrow, linewidth=2.0, label="Posterior - estreita")
52 ax.plot(x, post_wide, linewidth=2.0, label="Posterior - larga")
53 ax.axvline(z, linestyle="-.", linewidth=1.0, label=f"Medicao z = {z:.1f}")
54 ax.set_xlabel("Estado")
55 ax.set_ylabel("Densidade / escala relativa")
56 ax.set_title("Questao 9 - Efeito da largura da likelihood")
57 ax.grid(True, alpha=0.25)
58 ax.legend(fontsize=8)
59 fig.tight_layout()
60 fig.savefig(OUT_DIR / "questao9_update.pdf", bbox_inches="tight")
61 fig.savefig(OUT_DIR / "questao9_update.png", dpi=220, bbox_inches="tight")
62 plt.close(fig)
63
64 print(f"Prior:          mean={mean_prior:.4f}, std={std_prior:.4f}")
65 print(f"Posterior estreita: mean={mean_narrow:.4f}, std={std_narrow:.4f}")
66 print(f"Posterior larga:   mean={mean_wide:.4f}, std={std_wide:.4f}")

```

Listing 2: Comparação entre likelihood estreita e larga.

10. Simulação completa de um filtro de Bayes 1D

Enunciado. Simule

$$x_t = x_{t-1} + u + \epsilon, \quad z_t = x_t + \delta,$$

e compare o estado real com o estado estimado.

Solução. Foi implementado um filtro de Bayes em uma grade 1D. A cada passo o filtro faz duas operações:

- 1) **Predição:** usa o comando e o ruído de movimento para prever o novo estado;
- 2) **Correção:** usa a nova medição para ajustar essa previsão.

Foram usados

$$u = 0.80, \quad \sigma_\epsilon = 0.45, \quad \sigma_\delta = 1.00,$$

em 30 passos. Também foi usada uma semente aleatória fixa para que a simulação possa ser repetida com os mesmos resultados.

A estimativa mostrada no gráfico é a média do belief posterior.

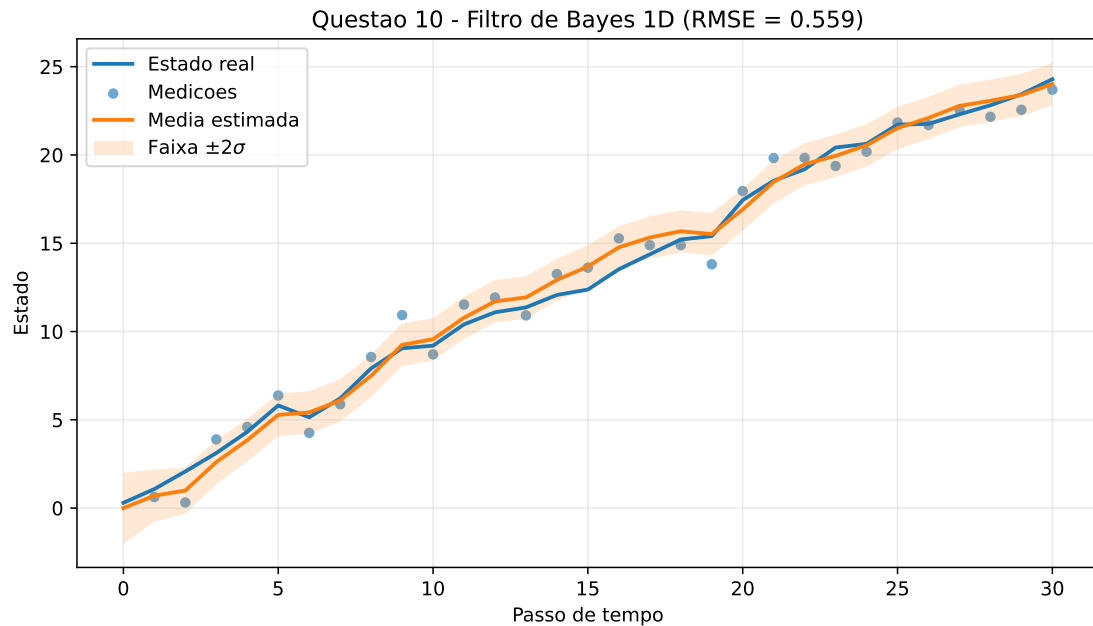


Figura 3: Estado real, medições e estimativa obtida pelo filtro de Bayes.

O erro quadrático médio da estimativa foi

$$\text{RMSE} \approx 0.559.$$

No último instante, o estado real foi 24.2905 e a estimativa foi 24.0177. Pelo gráfico, as medições são bem mais ruidosas, enquanto a estimativa acompanha o estado real de forma mais suave.

Esse é justamente o objetivo do filtro: **combinar o modelo de movimento com as medições para obter uma estimativa melhor do estado.**

10.1. Código da questão 10

```

1  """Questao 10 - Simulacao completa de um filtro de Bayes 1D.
2
3  Modelo de processo:  $x_t = x_{t-1} + u + \text{epsilon}$ .
4  Modelo de medicao:  $z_t = x_t + \text{delta}$ .
5  O belief e representado em uma grade e atualizado por convolucao + Bayes.
6  """
7
8  from pathlib import Path
9
10 import matplotlib.pyplot as plt
11 import numpy as np
12

```

```
13
14 OUT_DIR = Path(__file__).resolve().parents[1] / "imagens"
15 OUT_DIR.mkdir(parents=True, exist_ok=True)
16
17
18 def gaussian(x: np.ndarray, mean: float, std: float) -> np.ndarray:
19     return np.exp(-0.5 * ((x - mean) / std) ** 2) / (np.sqrt(2.0 * np.pi) * std)
20
21
22 def normalize_density(p: np.ndarray, dx: float) -> np.ndarray:
23     total = np.sum(p) * dx
24     if total <= 0.0:
25         raise RuntimeError("The density has zero numerical mass.")
26     return p / total
27
28
29 def moments(x: np.ndarray, p: np.ndarray, dx: float) -> tuple[float, float]:
30     mean = np.sum(x * p) * dx
31     var = np.sum((x - mean) ** 2 * p) * dx
32     return float(mean), float(np.sqrt(var))
33
34
35 rng = np.random.default_rng(935)
36
37 # Simulation parameters.
38 steps = 30
39 u = 0.80
40 sigma_process = 0.45
41 sigma_measurement = 1.00
42
43 # Wide grid so boundary effects are negligible during the simulation.
44 x_grid = np.linspace(-10.0, 40.0, 2501)
45 dx = x_grid[1] - x_grid[0]
46
47 # Initial belief and true state.
48 bel = gaussian(x_grid, mean=0.0, std=1.0)
49 bel = normalize_density(bel, dx)
50 x_true = 0.30
51
52 # Discrete convolution kernel for an increment u + epsilon.
53 offsets = (np.arange(x_grid.size) - x_grid.size // 2) * dx
54 motion_kernel = gaussian(offsets, mean=u, std=sigma_process)
55 motion_kernel = normalize_density(motion_kernel, dx)
56
57 true_history = [x_true]
58 measurement_history = [np.nan]
59 estimate_history = [moments(x_grid, bel, dx)[0]]
```

```

60 std_history = [moments(x_grid, bel, dx)[1]]
61
62 for _ in range(steps):
63     # Generate the true process and the noisy measurement.
64     x_true = x_true + u + rng.normal(0.0, sigma_process)
65     z = x_true + rng.normal(0.0, sigma_measurement)
66
67     # Prediction by convolution with the motion model.
68     bel_pred = np.convolve(bel, motion_kernel, mode="same") * dx
69     bel_pred = normalize_density(bel_pred, dx)
70
71     # Correction using the measurement likelihood.
72     likelihood = gaussian(x_grid, mean=z, std=sigma_measurement)
73     bel = normalize_density(likelihood * bel_pred, dx)
74
75     mean, std = moments(x_grid, bel, dx)
76     true_history.append(x_true)
77     measurement_history.append(z)
78     estimate_history.append(mean)
79     std_history.append(std)
80
81 true_history = np.asarray(true_history)
82 measurement_history = np.asarray(measurement_history)
83 estimate_history = np.asarray(estimate_history)
84 std_history = np.asarray(std_history)
85 t = np.arange(steps + 1)
86
87 rmse = np.sqrt(np.mean((estimate_history - true_history) ** 2))
88
89 fig, ax = plt.subplots(figsize=(8.2, 4.8))
90 ax.plot(t, true_history, linewidth=2.0, label="Estado real")
91 ax.scatter(t[1:], measurement_history[1:], s=20, alpha=0.65, label="Medicoes")
92 ax.plot(t, estimate_history, linewidth=2.0, label="Media estimada")
93 ax.fill_between(
94     t,
95     estimate_history - 2.0 * std_history,
96     estimate_history + 2.0 * std_history,
97     alpha=0.18,
98     label=r"Faixa $\pm 2\sigma$",
99 )
100 ax.set_xlabel("Passo de tempo")
101 ax.set_ylabel("Estado")
102 ax.set_title(f"Questao 10 - Filtro de Bayes 1D (RMSE = {rmse:.3f})")
103 ax.grid(True, alpha=0.25)
104 ax.legend()
105 fig.tight_layout()
106 fig.savefig(OUT_DIR / "questao10_simulacao.pdf", bbox_inches="tight")

```

```
107 fig.savefig(OUT_DIR / "questao10_simulacao.png", dpi=220, bbox_inches="tight")
108 plt.close(fig)
109
110 print(f"RMSE da media estimada: {rmse:.6f}")
111 print(f"Estado real final:      {true_history[-1]:.6f}")
112 print(f"Estimativa final:      {estimate_history[-1]:.6f}")
113 print(f"Desvio posterior final: {std_history[-1]:.6f}")
```

Listing 3: Filtro de Bayes 1D completo.